

Rukn — Final Project Report

CMPE 372 — USER INTERFACE DESIGN

FINAL PROJECT SUBMISSION: COMPLETE UI/UX & SYSTEM-AWARE INTERFACE DESIGN REPORT

Project Name: Rukn (رُكن) — Your Cozy Bilingual Reading & Journaling Corner
Course: CMPE 372 — User Interface Design, Section 02 (Computer Engineering)
Submission Date: June 4, 2026
Group Name: Group 02 (CMPE)

Group Members

Name	Student ID	Department
Zomorodah Bakhit	122200145	Computer Engineering
Omar Othman	122200162	Computer Engineering
Rewan Afifi	122200013	Computer Engineering
Reem Khalil	122200137	Computer Engineering
Faysal Dabbagh	122200015	Computer Engineering
Omar Anouti	121200031	Computer Engineering

1. Project Overview

1.1 Problem Definition

In the modern digital environment, readers face fragmented workflows and cognitive fatigue. Reading digital literature, taking reflective notes, and curating a focused environment are treated as separate, disconnected activities. Users read PDFs/EPUBs on one application, record notes in another, and play ambient sounds on a third. This fragmentation leads to:

- **Workflow Disruption:** Constant context switching between reading files and note-taking apps increases cognitive load and disrupts deep focus.
- **Loss of Context:** Reflections and quotes become detached from the specific pages or passages that inspired them, making retrieval difficult.
- **Distraction-Heavy Interfaces:** Standard reading applications lack sensory customization, leaving users exposed to notifications and digital fatigue.
- **Poor Bilingual Support:** Readers of right-to-left (RTL) languages like Arabic face misaligned text boxes, broken fonts, and jarring direction changes when writing mixed bilingual notes.

1.2 Target Users

Rukn serves a diverse cohort of focused readers:

- **Academic Students:** Need to digest course readings, capture citations, maintain reading streaks, and study under controlled conditions (e.g., Pomodoro timing with brown noise).
- **Avid General Readers:** Value visual aesthetics, require a cozy environment to unwind, and wish to keep a private journal of reflections alongside their library.
- **Bilingual Scholars & Translators:** Read and annotate classical literature in both English and Arabic, requiring native LTR and RTL rendering.

1.3 System Purpose

Rukn (رُكن — Arabic for *a corner*) is a bilingual web application designed to unite three experiences into one cozy space:

1. **Interactive e-book reader** for PDF and EPUB files.
2. **Passage-linked journaling system** (select text, highlight, and attach notes directly to passages).
3. **Ambient workspace** with illustrated backgrounds, procedural audio controls, and focus timers.

1.4 Value Proposition

Rukn provides a high-fidelity, distraction-free environment that integrates reading, journaling, and environmental customization. By using sub-pixel typography, glassmorphic UI elements, and a browser-synthesized ambient audio engine, it turns reading into an immersive sensory experience.

1.5 Project Scope

The scope covers:

- **User Accounts:** Multi-tier access controls (Guest, Free Registered, and Paid Premium), with full bcrypt-hashed authentication, a one-hour token forgot-password flow, and a JSON account-data export.
- **Personal Library:** Hand-drawn and procedural book covers, status progress bars, format/lang badges, search, and tab filtering across My Books, Public Domain, and Collections.
- **Linked & Freeform Journals:** Notes attached to passage selections, freeform reflections, hashtag tagging with automatic tag garbage-collection when their last entry is deleted.
- **Procedural Ambience Controller:** Custom Web Audio generators for rain, fireplace, café, forest, lofi (with optional MP3 override), plus a 10/25/50/60-minute focus timer.

1.6 Why the problem matters

Reading is a fundamental learning activity, yet the surrounding workflow has fractured. Students copy passages into Notes, jump to Spotify for ambient sound, set a timer in iOS Clock, and then write notes in a fourth app — losing context with every switch. National Library of Medicine studies on context-switching place the cognitive recovery cost at 23 minutes per disruption. For undergraduates juggling 4–6 simultaneous textbooks, that compounds into hours of lost focus per week. Rukn matters because it dissolves these silos into one workspace: read, highlight, reflect, and shape your environment in the same tab — so the act of reading stays whole.

2. Personas

We designed three core personas to guide our design, interface behaviors, and database structure.

Persona 1: Elif Kaya (Student User)

- **Age:** 21
- **Role:** Undergraduate Computer Engineering Student
- **Technology Comfort:** High
- **Goals:**
 - Quickly upload and organize course PDFs.
 - Keep key textbook highlights linked directly to pages.
 - Study in a distraction-free, aesthetically calming dashboard.
- **Frustrations:**
 - Losing quotes when copying them into separate document files.
 - Getting distracted by tab switching and notifications during study sessions.
 - Flat, uninspiring user interfaces.
- **Digital Behavior:** Uses a laptop and smartphone simultaneously; relies on Obsidian for markdown notes; plays lo-fi streams in the background.
- **Typical Tasks:**
 - Uploading lecture slides.
 - Highlighting key exam topics.
 - Studying with a 25-minute Pomodoro timer.
- **System Needs:** Easy file uploads, search-indexed tags, and integrated audio cues.

Persona 2: Mehmet Arslan (Academic Researcher & Translator)

- **Age:** 38
- **Role:** Literature Translator & Academic Writer
- **Technology Comfort:** Medium
- **Goals:**
 - Read classical Arabic poetry alongside English translations.
 - Write complex bilingual notes without layout glitches.
 - Tag entries by themes (e.g., #wisdom, #metaphor).
- **Frustrations:**
 - Jarring layout shifts when mixing Arabic and English text in the same file.
 - Inability to locate the exact page a quote came from.
- **Digital Behavior:** Works primarily on desktop; prefers clean serif typography; keeps archives of text files.
- **Typical Tasks:**
 - Reading bilingual texts side-by-side.
 - Extracting quotes and appending translation notes.
 - Exporting data for research drafts.
- **System Needs:** High-contrast serif fonts, RTL/LTR bi-directional text support, and passage-linked notes.

Persona 3: Ahmet Demir (Campus Admin / Premium User)

- **Age:** 45
- **Role:** Campus Supervisor & Avid Reader
- **Technology Comfort:** Medium
- **Goals:**
 - Listen to audiobooks during commutes.
 - Curate specialized reading collections.
 - Access books and notes offline.
- **Frustrations:**
 - Having to pay multiple subscriptions for audio, books, and focus apps.
 - Complicated, busy layouts with small, unreadable text.
- **Digital Behavior:** Reads on tablets; uses voice-to-text; values simple, clear navigation.
- **Typical Tasks:**
 - Creating custom folders (Collections).

- Listening to audiobook narration.
- Upgrading preferences for dark reading modes.
- **System Needs:** Clean visual structure, large touch targets, premium audio playback, and account settings data export.

3. User Stories

Our system accommodates different user roles (Guest, Free User, Paid User) through the following stories:

- **US1 (Guest):** *As a guest*, I want to preview the ambient sound slider and background timer on the homepage, *so that* I can sample the platform's focus environment before signing up.
- **US2 (Free User):** *As a registered student*, I want to create an account with my email and a unique username, *so that* my preferences, reading progress, and notes are saved.
- **US3 (Free User):** *As a reader*, I want to upload PDF and EPUB files to my library, *so that* I can read my personal collection within the app.
- **US4 (Free User):** *As a student*, I want to highlight text passages in a book and write notes attached to them, *so that* I can review my reflections in context later.
- **US5 (Free User):** *As a bilingual writer*, I want to write Arabic reflections that display correctly from right to left, *so that* I can take mixed-language notes without layout issues.
- **US6 (Free User):** *As a user*, I want to categorize my journal entries using hashtag-style tags, *so that* I can search and filter my reflections globally.
- **US7 (Premium User):** *As a premium member*, I want to cycle through ambient backgrounds (Maghrib Dusk, Olive Grove, Cozy Library) and adjust sound levels, *so that* I can customize my environment to my current mood.
- **US8 (Premium User):** *As an on-the-go reader*, I want to unlock audiobooks and listen to high-quality narration, *so that* I can consume my books when I cannot read.
- **US9 (System):** *As a system*, I want to validate passwords on the register form to ensure they are at least 8 characters long, *so that* user accounts remain secure.
- **US10 (System):** *As a system*, I want to automatically save reading progress every 30 seconds, *so that* the user never loses their page.
- **US11 (All Users):** *As a user*, I want to download all my account data in JSON format or delete my account, *so that* I have full control over my data.

4. User Journey Map

This journey map follows **Elif Kaya** through a study session using Rukn.

Stage	1. Discovery & Entry	2. Setting Up	3. Reading & Highlighting	4. Journaling	5. Completion
User Tasks	Opens the app, logs in, and views her dashboard.	Selects <i>The Prophet</i> from "Continue Reading" and opens it.	Reads Chapter 3, highlights a quote on page 47, and plays rain audio.	Opens the note panel, types a reflection, and saves it.	Checks her updated reading stats and closes the app.
Actions	Enters username and password.	Clicks "Resume".	Drags mouse over text; clicks the "Ambience" chip.	Types thought; enters #wisdom tag; clicks "Save".	Views her 7-day streak update; logs out.
Emotions	Calmed by the warm dusk	Focused and ready to study.	Relaxed by the rain sound; engaged	Reflective; feels a sense of	Satisfied; motivated to

Stage	1. Discovery & Entry background.	2. Setting Up	3. Reading & Highlighting with the text.	4. Journaling accomplishment.	5. Completion return tomorrow.
Pain Points	Form validation errors can feel jarring.	Loading books can feel slow on weak connections.	Audio volume can start too loud.	Formatting tags manually can lead to typos.	Stat tracking might not count offline time.
Opportunities	Use soft red warnings instead of browser popups.	Implement skeleton loaders for book text.	Default master volume to a soft 40%.	Provide auto-completing tags as the user types.	Sync offline timestamps once reconnected.

5. Information Architecture (IA)

Our IA is designed to reduce cognitive load by grouping tasks into clear, role-based navigation layers.

5.1 Main Modules

Rukn divides the application into four orthogonal modules so each task lives in exactly one place:

1. **Home / Dashboard** — the cozy entry workspace where reading, journaling, and ambience all live together.
2. **Library** — book discovery, organization, and reading-progress overview.
3. **Journal** — every reflection ever written, searchable and filterable.
4. **Settings** — account control, preferences, plan, and data export.

5.2 Role-Based Navigation

The same four modules are visible to every signed-in user, but their content adapts to role:

- **Guest** — sees only the Homepage; clicking any deeper page bounces to Login.
- **Free Registered User** — full access to Home, Library, Journal, Settings. The Sounds and Customization widgets show "Locked — Try Premium" overlays for premium-only sounds.
- **Paid Premium User** — all locks removed; the Premium badge appears on the avatar; full palette set + every ambient sound enabled.

5.3 Primary Navigation

A persistent vertical icon rail on the left of every signed-in page. Items: **Home · Library · Journal · Settings**. The rail expands on hover to reveal the brand mark and labels.

5.4 Secondary Navigation (Module-Specific)

- **Library:** Tabs for *My Books*, *Public Domain*, *Collections*.
- **Journal:** Tabs for *All*, *By Book*, *Freeform*; a tag-filter dropdown; a sort pill (Newest / Oldest).
- **Settings:** Sidebar with sections *Account*, *Reading Preferences*, *Plan*, *Account Data*.
- **Reader:** Vertical tool rail (Highlight · Note · Bookmark · Focus · Music · Display) plus a bottom progress paginator.

5.5 Labeling System

Labels are written for readers, not engineers. We use "Save entry" instead of "Insert record", "Drag-select any passage to highlight" instead of "Create highlight from selection", and tier labels ("Free plan" / "Premium") rather than role IDs. Every microcopy choice was tested against the personas before committing.

5b. Site Map

The page hierarchy follows the four-module IA above. Public pages are reachable without a session; protected pages auto-redirect to Login.

Public area (no session)	
Homepage	→ public landing, sound preview dock
Login / Register	→ bcrypt-authenticated entry
Forgot Password	→ email a one-hour reset token
Reset Password	→ set a new password via the token link
Authenticated area	
Dashboard (Home)	← default after login
Continue Reading widget	→ resumes into Reader
Today's Journal widget	→ opens Journal on save
Reading Stats widget	→ 7-day bar + streak
Ambient & Focus dock	→ procedural audio + timer
Library	
My Books tab	→ in-progress + uploaded books
Public Domain tab	→ seeded public library
Collections tab	→ user-named folders
Book Card	→ Reader.html?book={id}
Reader (focused single-page view)	
Passage prose with inline highlights	
Highlight popover	→ save highlight (5 colors)
Add-note modal	→ cross-creates a journal entry
Right-side notes panel	→ all notes for this chapter
Journal	
Entries list (left pane)	→ filtered by search/tag
Entry detail (right pane)	→ edit / delete / share
New entry overlay	→ mood + tags + body
Settings	
Account	→ username, email, language, password
Reading Preferences	→ theme, font, font-size, ambient, volume
Plan	→ Free / Premium upgrade-downgrade
Account Data	→ sign out · export JSON · delete account

6. Page Content Inventory

We structured our 5 core screens with clear purposes, content lists, and user actions.

Page	Purpose	Main Content	User Actions
Login / Register	Authentication entry point	Split-column view, brand art vignette, tabbed form card, input fields.	Toggle tabs, validate inputs, hide/show password, submit.
Dashboard	Center of active reading sessions	Continue Reading card, Today's Journal composer, Stats summary, Ambient audio dock.	Resume reading, type entry, toggle sound chips, drag volume, cycle themes.

Page	Purpose	Main Content	User Actions
Library	Personal bookshelf manager	Grid of book covers, format badges, progress bars, search bar, sort/filter pills.	Click book to read, search, sort by title/author, create collection, upload file.
Reader View	Reading and annotation workspace	Centered sepia paper page, vertical tool rail, notes sidebar, progress footer.	Turn page, highlight text, add inline note, toggle timer, bookmark page.
Journal	Repository of personal reflections	Split two-pane grid: entries list (left) and selected note detail (right).	Search notes, filter by tag, delete entry, edit text, copy citation link.

7. High-Fidelity Prototype

The Rukn high-fidelity prototype consists of 8 main screens designed around the warm **Desert Dusk** palette:

- **Screen 01: Homepage (Maghrib):** Features a silhouette of domed mosques and minarets against a gradient sky. The bottom dock allows users to toggle ambient sounds and timers.
- **Screen 02: Login / Register:** A split layout. The left side displays a hanging lantern on a windowsill, and the right side hosts a clean, glassmorphic login card.
- **Screen 03: Dashboard:** Displays three glass widgets: *Continue Reading*, *Today's Journal*, and *Reading Stats*, floating over the dimmed sky background.
- **Screen 04: Library:** Displays hand-illustrated book covers with progress bars and format badges.
- **Screen 05: Reader:** A sepia paper layout with a toolbar on the left and annotations on the right.
- **Screen 06: Journal:** A two-pane layout showing list items on the left and full-page entries (including Arabic text) on the right.
- **Screen 07: Settings:** Contains sub-cards for updating credentials, cycling themes, and exporting data.
- **Screen 08: Premium:** Hosts pricing cards, monthly/yearly toggles, and feature highlights.

8. Accessibility Review (WCAG 2.1 AA Compliance)

Rukn was built from the ground up to meet accessibility standards:

- **Visual Accessibility:**
 - All text-to-background combinations maintain a contrast ratio of at least **4.5:1** (e.g., light-paper text #F6EDD8 on dark glass #140C07 yields a contrast of 11.2:1).
 - System alerts do not rely on color alone; error states pair a red border with a warning icon (⚠) and helper text.
- **Typography Readability:**
 - Display elements use *Cormorant Garamond*, while UI controls use *Manrope* for readability on screens.
 - Line heights are set to 1.5 for sans-serif text and 1.6 for reading paragraphs, preventing overlapping.
- **Keyboard Accessibility:**
 - All interactive buttons, chips, and input fields are focusable using the Tab key.
 - Focused elements display a high-contrast orange outline (outline: 2px solid var(--sky-sun)).
- **Cognitive Load Reduction:**
 - Widget panels are visually separated to group related information.
 - Form fields use clear helper hints (e.g., lowercase, letters/numbers/dash only for tags) to prevent input errors.

9. Dynamic UI & Interaction Logic

Our interface relies on real-time feedback and state transitions:

- **Form Validation Logic:**
 - **Client-side:** Password fields show warnings if they are under 8 characters. The email field validates against standard regex patterns.
 - **Visual Feedback:** Fields with errors display a red border (1px solid #E26B5A), and primary buttons are disabled (opacity 0.4, cursor: not-allowed) until all fields are valid.
- **Loading States:**
 - Skeleton cards are rendered on initial library loads to indicate content is loading.
- **Asynchronous CRUD Actions:**
 - Submitting a journal entry triggers a POST request. On success, the new entry is prepended to the list with a fade-in animation, and the entry count updates without reloading the page.
- **Ambient Customization:**
 - Clicking a sound chip triggers the sound and updates the database via an async API request.

10. Database Design (MySQL ER Logic)

10.1 High-Level Core Entities

1. **User:** Represents registered members. Crucial for identifying data ownership and access tiers.
2. **UserPreferences:** Stores workspace themes, fonts, and active sound settings in a 1:1 relationship with users.
3. **Book:** Holds book metadata, file locations, and formats.
4. **ReadingProgress:** Tracks user progress (pages read, time spent) for each book.
5. **Journal:** The parent container for reflections (either freeform or book-specific).
6. **JournalEntry:** The written thoughts, optionally linked to a highlight.
7. **Tag:** Categorization labels created by users.

10.2 Database Normalization

- **1NF:** All column values are atomic. Multivalued lists (like tags) are moved to a separate junction table (journal_entry_tags).
- **2NF:** All non-key attributes are fully dependent on the primary keys. Composite keys are used only in junction tables (collection_books, journal_entry_tags), where no partial dependencies exist.
- **3NF:** Non-key attributes depend only on the primary key, eliminating transitive dependencies. For example, preferences are stored in user_preferences rather than users to prevent dependencies on non-key columns.

10.3 Relationships, Cardinality, and Foreign Keys

Relationship	Cardinality	PK → FK
users → free_users / paid_users	1 : 0..1	users.user_id → free_users.user_id / paid_users.user_id (ISA)
users → user_preferences	1 : 1	users.user_id → user_preferences.user_id (UNIQUE)
users → password_resets	1 : N	users.user_id → password_resets.user_id

Relationship	Cardinality	PK → FK
users → books (uploaded)	1 : N	users.user_id → books.uploaded_by (NULL on user delete)
users → reading_progress	1 : N	users.user_id → reading_progress.user_id
books → reading_progress	1 : N	books.book_id → reading_progress.book_id (UNIQUE per user+book)
users → highlights	1 : N	users.user_id → highlights.user_id
books → highlights	1 : N	books.book_id → highlights.book_id
users → journals	1 : N	users.user_id → journals.user_id
journals → journal_entries	1 : N	journals.journal_id → journal_entries.journal_id
journal_entries ↔ tags	M : N (via journal_entry_tags)	composite PK (entry_id, tag_id)
users → collections	1 : N	users.user_id → collections.user_id
collections ↔ books	M : N (via collection_books)	composite PK (collection_id, book_id)

All FKs use ON DELETE CASCADE for personal data (preferences, journals, highlights, bookmarks, reading progress) so deleting a users row cleans everything up. The only exception is books.uploaded_by, which uses ON DELETE SET NULL to preserve community-uploaded books even if their original uploader is deleted.

10.4 Business Rules & Constraints

- Tier Exclusivity** — A user can be in either free_users or paid_users, never both. Upgrading is an atomic transaction: DELETE FROM free_users WHERE user_id = ? + INSERT INTO paid_users (user_id) VALUES (?).
- One Progress Row Per Book** — A user can have at most one reading_progress row per book, enforced by UNIQUE KEY uq_progress_user_book(user_id, book_id). Re-opening a book updates rather than duplicates.
- Exactly One Freeform Journal** — Each user gets a single type='freeform' journal at registration (via the same transaction that creates the user). Book-linked journals require a non-NULL book_id and are unique per (user_id, book_id) pair.
- Atomic Account Creation** — Registration writes to users, free_users, user_preferences, and journals inside a single transaction. If any insert fails, all four are rolled back.
- Password Reset Token Expiry** — Tokens in password_resets are 64-char hex strings with a 1-hour expires_at and a used BOOLEAN. The reset endpoint only accepts unexpired, unused tokens.
- Auto-GC for Tags** — Deleting the last journal_entry that referenced a tag triggers a cleanup query that removes the now-orphaned tags row, keeping the user's tag list tidy.
- Input Validation at the DB Level** — users.email is UNIQUE; users.username is UNIQUE; highlights.color has a CHECK constraint allowing only ('yellow', 'green', 'blue', 'pink'); journals.type accepts only ('book', 'freeform').

11. UI–Database Mapping (CRUD Matrix)

This matrix maps user actions on screens to specific database tables and CRUD operations.

Screen / Action	Database Table	CRUD Action	SQL Operation
Register User	users, free_users, user_preferences, journals	Create	INSERT INTO users... (transactional)
Login User	users	Read	SELECT * FROM users WHERE email = ?
Load Continue Reading	reading_progress join books	Read	SELECT * FROM reading_progress JOIN books...
Auto-Save Page Progress	reading_progress	Update	UPDATE reading_progress SET current_page = ?
Save Journal Entry	journal_entries	Create	INSERT INTO journal_entries...
Delete Journal Entry	journal_entries	Delete	DELETE FROM journal_entries WHERE entry_id = ?
Toggle Ambient Sound	user_preferences	Update	UPDATE user_preferences SET ambient_sound = ?
Upgrade to Premium	paid_users, free_users	Create & Delete	INSERT INTO paid_users... DELETE FROM free_users

12. Sample SQL Logic

12.1 SELECT Queries (Filtering, Aggregating, Grouping, & Joins)

1. Retrieve active reading targets for a user's dashboard

```

SELECT
    b.book_id,
    b.title,
    b.author,
    b.cover_image,
    b.total_pages,
    rp.current_page,
    rp.pages_read,
    ROUND((rp.pages_read / b.total_pages) * 100, 1) AS percent_complete
FROM reading_progress rp
JOIN books b ON rp.book_id = b.book_id
WHERE rp.user_id = 7 AND rp.is_completed = FALSE
ORDER BY rp.last_read_at DESC
LIMIT 1;

```

2. Aggregate total reading hours and finished book counts for a user

```

SELECT
    COUNT(DISTINCT book_id) AS total_books_started,
    SUM(CASE WHEN is_completed = TRUE THEN 1 ELSE 0 END) AS total_books_finished,
    ROUND(SUM(total_seconds_read) / 3600.0, 1) AS total_hours_read
FROM reading_progress
WHERE user_id = 7;

```

3. Fetch recent journal entries along with their tags

```

SELECT
    je.entry_id,

```

```

je.mood,
je.content,
je.created_at,
GROUP_CONCAT(t.name ORDER BY t.name SEPARATOR ', ') AS tags
FROM journal_entries je
JOIN journals j ON je.journal_id = j.journal_id
LEFT JOIN journal_entry_tags jet ON je.entry_id = jet.entry_id
LEFT JOIN tags t ON jet.tag_id = t.tag_id
WHERE j.user_id = 7 AND j.type = 'freeform'
GROUP BY je.entry_id
ORDER BY je.created_at DESC;

```

12.2 INSERT Queries

```

-- Create a new user record
INSERT INTO users (user_id, username, email, password_hash, preferred_lang)
VALUES (8, 'omar_dev', 'omar@example.com', '$2y$10$hashedstring...', 'EN');

-- Insert default workspace preferences for a user
INSERT INTO user_preferences (user_id, theme, background, font, font_size, ambient_sound,
    sound_volume)
VALUES (8, 'dusk', 'plain', 'Lora', 16, 'fireplace', 0.40);

-- Save a highlighted passage
INSERT INTO highlights (user_id, book_id, page_number, selected_text, color)
VALUES (7, 14, 47, 'Love has no other desire but to fulfill itself.', 'yellow');

```

12.3 UPDATE Queries

```

-- 1. Advance reading progress one page
UPDATE reading_progress
SET current_page      = current_page + 1,
    pages_read        = pages_read + 1,
    total_seconds_read = total_seconds_read + 30,
    last_read_at       = NOW()
WHERE user_id = 7 AND book_id = 14;

-- 2. Save workspace preferences (palette + ambience + volume)
UPDATE user_preferences
SET theme          = 'olive',
    ambient_sound  = 'rain',
    sound_volume   = 0.65,
    font           = 'Lora',
    font_size      = 17
WHERE user_id = 7;

-- 3. Mark a journal entry as edited (touching updated_at via ON UPDATE CURRENT_TIMESTAMP)
UPDATE journal_entries
SET content = 'Re-reading at 30 hits differently than at 20.',
    mood    = '🌧️'
WHERE entry_id = 1001
    AND journal_id IN (SELECT journal_id FROM journals WHERE user_id = 7);

-- 4. Burn a password-reset token after a successful reset
UPDATE password_resets
SET used = TRUE
WHERE token = ?
    AND used = FALSE
    AND expires_at > NOW();

```

12.4 DELETE Queries

```
-- 1. Delete a specific journal entry (cascades to junction table)
DELETE FROM journal_entries
  WHERE entry_id = 1001
        AND journal_id IN (SELECT journal_id FROM journals WHERE user_id = 7);

-- 2. Garbage-collect tags that no longer reference any entry (after entry delete)
DELETE FROM tags
  WHERE user_id = 7
        AND tag_id NOT IN (SELECT DISTINCT tag_id FROM journal_entry_tags);

-- 3. Cascade delete a user account (every dependent row vanishes)
DELETE FROM users WHERE user_id = 8;
```

12.5 JOIN Queries (cross-entity reads)

```
-- 1. Build the dashboard "Continue Reading" tile
SELECT b.title, b.author, b.total_pages,
       rp.current_page, rp.pages_read,
       ROUND((rp.pages_read / b.total_pages) * 100, 0) AS pct
  FROM reading_progress rp
  JOIN books b ON rp.book_id = b.book_id
 WHERE rp.user_id = 7 AND rp.is_completed = FALSE
 ORDER BY rp.last_read_at DESC
 LIMIT 1;

-- 2. List every journal entry with its tags concatenated
SELECT je.entry_id, je.mood, je.content, je.created_at,
       GROUP_CONCAT(t.name ORDER BY t.name SEPARATOR ', ') AS tags
  FROM journal_entries je
  JOIN journals j      ON je.journal_id = j.journal_id
  LEFT JOIN journal_entry_tags jet ON je.entry_id = jet.entry_id
  LEFT JOIN tags t      ON jet.tag_id = t.tag_id
 WHERE j.user_id = 7
 GROUP BY je.entry_id
 ORDER BY je.created_at DESC;

-- 3. Pull all highlights a user made in The Prophet, with the book title
SELECT h.page_number, h.selected_text, h.color, h.created_at, b.title
  FROM highlights h
  JOIN books b ON h.book_id = b.book_id
 WHERE h.user_id = 7 AND b.title = 'The Prophet'
 ORDER BY h.page_number;
```

13. Front-End & Backend Implementation

Rukn is implemented end-to-end with a vanilla React (Babel-standalone, no build step) frontend talking to a PHP 8 + MySQL backend over a same-origin JSON API. All seven screens are interactive and persist their state through the backend.

13.1 Folder layout

```
RUKN/
├── *.html          ← 9 pages (Homepage, Login, Forgot, Reset,
```

	Dashboard, Library, Journal, Reader, Settings)
*.jsx	← React components, served as-is
styles.css	← 3,300-line design system
api/	
db.php	← PDO connection
auth.php	← me / register / login / logout / forgot / reset
api.php	← 14 CRUD endpoints (JSON)
upload.php	← multipart PDF/EPUB upload
client.js	← fetch wrapper exposed as window.RuknAPI
audio.js	← procedural Web Audio engine

13.2 Backend stack

- **api/db.php** — PDO connection to MySQL with `ATTR_ERRMODE = ERRMODE_EXCEPTION` and `ATTR_EMULATE_PREPARES = false`. Every query is a parameterized prepared statement; nothing is ever string-concatenated into SQL.
- **api/auth.php** — bcrypt password hashing via `password_hash($pw, PASSWORD_DEFAULT)`. Registration runs four inserts (users + free_users + user_preferences + journals) inside a single transaction so partial failure rolls back cleanly. Sessions are server-side via `session_start()`; the cookie is `httponly`.
- **api/api.php** — 14 JSON endpoints covering dashboard, library, journal_list, save_entry, delete_entry, settings_get, update_preference, upgrade, downgrade, export, delete_account, save_highlight, list_highlights, advance_progress.
- **api/upload.php** — accepts multipart POST, 20 MB cap, extension-checked (.pdf / .epub), stores files under `RUKN/uploads/<book_id>_<safe-name>.<ext>`.

13.3 Frontend stack

- **React 18** via Babel-standalone — no Webpack, no Vite, no build step. Just `<script type="text/babel" src="...jsx">` tags. This was a deliberate choice for grading reproducibility: the grader doesn't need npm or Node, only XAMPP.
- **api/client.js** — a 60-line wrapper around `fetch()` that auto-includes session cookies, JSON-encodes bodies, and bounces protected pages to `Login.html` on 401. Exposed as `window.RuknAPI`.
- **api/audio.js** — 200 lines of Web Audio: noise buffers, biquad filters, oscillators, gain envelopes. Five procedural sounds (rain, fireplace, café, forest, lofi) that loop seamlessly with no MP3 assets. If an MP3 with the matching name is dropped into `RUKN/audio/`, the engine plays it instead.

13.4 At-least-one HTML+CSS+JS screen (rubric req)

`RUKN/Login.html` is the canonical example. It loads `auth.jsx`, which contains:

- **HTML5 validation attributes** — `required`, `autocomplete`, `type="email"`, `type="password"`
- **JS validation engine** — on every keystroke, the registration form re-evaluates four rules: username regex `^[a-z0-9_]{3,20}$`, email regex, password ≥ 8 chars, confirm-password match. Failing rules add a `has-error` class to the field, render an inline error message, and disable the Submit button.
- **Async fetch** — submit calls `RuknAPI.register({...})` which POSTs JSON to `/RUKN/api/auth.php?action=register`. The response sets `setServerError(...)` on failure (rendered as a red banner inside the auth card) or redirects to `Dashboard.html` on success.
- **Dynamic UI** — tab switcher (Login ↔ Create account), password show/hide eye toggle, sliding tab indicator pill, language pill (display-only), automatic redirect to Dashboard if a session already exists on mount.

13.5 Source code

- **GitHub repo:** <https://github.com/ZomorodahBakhit/rukun>

- **One-click launcher:** double-click `start.bat` (auto-detects XAMPP, starts MySQL, imports schema if needed, starts a PHP server on `localhost:8000`, opens the browser).
 - **Detailed setup with an alternative XAMPP-htdocs path:** see [SETUP.md](#).
-

14. Final Reflection

14.1 Biggest Challenge

Our biggest challenge was implementing the **Procedural Web Audio Engine** in vanilla JavaScript. Generating soundscapes directly in the browser required precise scheduling of noise buffers, oscillators, and filters. Getting these components to loop smoothly without pops or clicks while mapping volume sliders dynamically was a significant hurdle.

14.2 What Changed During the Project

Initially, we planned to focus solely on reading progress and notes. However, user research revealed that **sensory environment** was a key factor in study retention. As a result, we expanded the scope of our dashboard to integrate sound, visual themes, and timers as core widgets, creating a more comprehensive study corner.

14.3 UI/UX and Systems Thinking Learnings

This project taught us that UI design goes beyond aesthetics. By mapping user actions to database queries, we learned to treat interfaces as part of a larger system. Every button toggle, form field, and transition must align with database constraints and error-handling routines, showing how design decisions directly affect backend architecture.